

Containers e Docker na Arquitetura de Nuvem

João Cardoso

23 de julho de 2026

O Problema

O Problema: Cozinhas Diferentes, Mesmo Prato

Analogia: Cozinha

- **Receita:** Seu código
- **Ingredientes:** Bibliotecas e dependências
- **Fogão:** SO (Windows, Linux, macOS)
- **Utensílios:** Versões do Python/Node/Java

Se cada cozinha tem equipamentos diferentes, o mesmo prato pode dar errado!

No Desenvolvimento...

- Dev usa Windows com Python 3.11 → *Roda!*
- Servidor usa Linux com Python 3.8 → *Erro!*
- Colega usa macOS com Python 3.9 → *Quebra!*

Container = Kit completo de cozinha portátil!

Leva seus próprios ingredientes e utensílios para qualquer lugar.

VM vs Container

VM vs Container: Diferenças de Arquitetura

Máquina Virtual (Pesada)

A P L I C A Ç Ã O	← Seu software
B I B L I O T E C A S	← Python, libs, dependências
S O C O N V I D A D O	← Cada VM = 1 SO!
(CADA VM TEM O SEU)	
H Y P E R V I S O R	← VMware, KVM, Hyper-V
S O A N F I T R I Ã O	← Windows Server, Linux
H A R D W A R E	← CPU, RAM, Disco

PESADO: GBs de espaço, minutos para iniciar

Container (Leve)

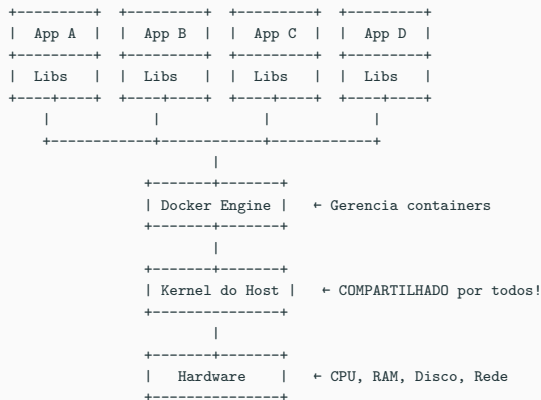
A P L I C A Ç Ã O	← Seu software
B I B L I O T E C A S	← Dependências ISOLADAS
D O C K E R E N G I N E	← Gerencia containers
S O A N F I T R I Ã O	← ÚNICO SO COMPARTILHADO
(ÚNICO SO PARA TODOS)	
H A R D W A R E	← CPU, RAM, Disco

LEVE: MBs de espaço, segundos para iniciar

Diferença fundamental: VM tem SO completo por instância; Container compartilha o kernel do host.

Containers: O Coração da Reprodutibilidade

Como os containers compartilham o mesmo Kernel do Host OS, eles são **leves** e **rápidos**, mas mantêm **isolamento** entre si através de namespaces e cgroups.



- **Isolamento**: Cada container tem seu próprio sistema de arquivos, rede e processos
- **Leveza**: Sem SO convidado = economia de GBs de espaço e RAM
- **Velocidade**: Inicialização em segundos (vs. minutos das VMs)
- **Portabilidade**: Funciona em qualquer lugar com Docker instalado → compatibilidade em qualquer máquina

Docker na Prática

Dockerfile: A “Receita” do Ambiente

```
1 FROM python:3.10 - slim      # Define a Imagem Base (Leve)
   WORKDIR /app                # Define Diretorio de Trabalho
3 RUN apt-get update && ...     # Instala dependencias do Sistema
   COPY requirements.txt .      # Copia lista de bibliotecas
5 RUN pip install -r req.txt    # Instala dependencias Python
   COPY src/ ./src/            # Copia seuCodigo Fonte
7 ENTRYPOINT ["python"]        # Comando de Execucao Fixo
```

Dica: Use *multi-stage builds* para reduzir o tamanho da imagem final, separando o estágio de compilação do de execução.

Exemplo Prático

Objetivo

Construir o “Hello, World!” de Python no Docker.

O que é um Dockerfile?

Um **Dockerfile** é um arquivo de texto com instruções para construir uma imagem Docker.

- **Receita**: Define o ambiente e os passos para criar a imagem
- **Instruções**: Comandos como FROM, COPY, RUN, CMD
- **Reprodutível**: Garante que a imagem seja construída da mesma forma sempre
- **Versionável**: Pode ser armazenado no Git como qualquer código

O que é uma API Flask?

Flask é um **micro-framework** para Python que permite criar aplicações web e APIs REST de forma simples e rápida.

- **Rotas:** `@app.route('/')` define endpoints
- **Requisições:** GET, POST, PUT, DELETE
- **Respostas:** JSON, HTML, texto plano
- **Leveza:** Ideal para microserviços e protótipos

Flask é usado por empresas como Netflix, Uber e Airbnb.

Exemplo: Containerizando uma API Flask

1. Código (app.py)

```
1 from flask import Flask, jsonify
2 app = Flask(__name__)           # Cria a aplicação
3
4 @app.route('/')                 # Rota principal
5 def hello():
6     return "Hello, Docker!"    # Resposta em texto
7
8 @app.route('/health')          # Rota de health check
9 def health():
10     # Resposta em JSON com status 200
11     return jsonify({"status": "ok"}), 200
12
13 if __name__ == '__main__':
14     # host='0.0.0.0' permite acesso externo
15     app.run(host='0.0.0.0', port=5000)
```

2. Dockerfile

```
1 FROM python:3.10 - slim          # Imagem base leve
   WORKDIR /app                    # Diretório de trabalho
3 COPY requirements.txt .           # Cópia dependências
   RUN pip install -r requirements.txt # Instala dependências
5 COPY app.py .                    # Cópia código
   EXPOSE 5000                     # Expõe a porta 5000
7 CMD ["python", "app.py"]         # Comando de execução
```

3. Execução Passo a Passo

```
1 # Build da imagem
  docker build -t minha-api .
3
  # Executar container em background
5 docker run -d -p 5000:5000 --name api minha-api

7 # Testar a API
  curl http://localhost:5000
9 # Hello , Docker!

11 # Testar health check
  curl http://localhost:5000/health
13 # {"status": "ok"}

15 # Ver logs
  docker logs api
17
  # Parar e remover
19 docker stop api && docker rm api
```

Desafio

Desafio: Containerize seu Próprio Código

Instruções

1. Escolha um programa simples em Python que você já tenha feito (pode testar fazer com um novo, também)
2. Containerize-o seguindo o modelo do exemplo
3. Envie:
 - Dockerfile
 - Código fonte
 - requirements.txt (se houver)
 - Print da execução

Desafio: Containerize seu Próprio Código

Critérios de Avaliação

Critério	Peso
Dockerfile correto	40%
Código funcional no container	40%
Documentação/prints	20%
Criatividade (bônus)	10%

Prazo: 1 semana

Conclusão

- **ECR**: Armazena suas imagens Docker com segurança.
- **AWS Batch**: Orquestra e executa milhares de containers em paralelo.
- **S3**: Armazena os resultados finais das simulações.



Conclusão: Containers garantem reprodutibilidade; a Nuvem garante escala.

OBRIGADO!!